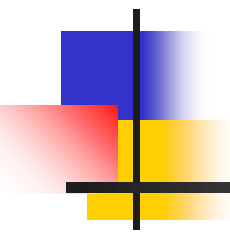
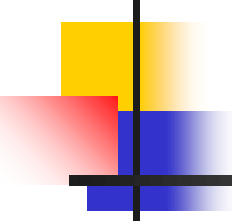


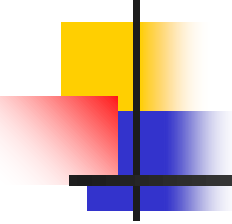
Properties of Regular Languages





Topics

- 1) How to prove whether a given language is regular or not?
- 1) Closure properties of regular languages
- 1) Minimization of DFAs



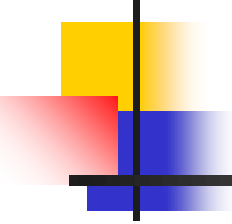
Some languages are *not* regular

When is a language is regular?

if we are able to construct one of the following: DFA *or* NFA *or* ϵ -NFA *or* regular expression

When is it not?

If we can show that no FA can be built for a language



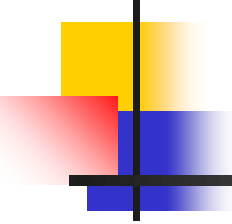
How to prove languages are *not* regular?

What if we cannot come up with any FA?

- A) Can it be language that is not regular?
- B) Or is it that we tried wrong approaches?

How do we *decisively* prove that a language is not regular?

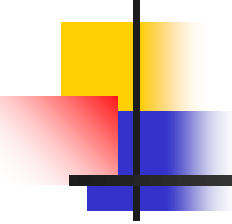
“The hardest thing of all is to find a black cat in a dark room, especially if there is no cat!” -Confucius



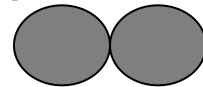
Example of a non-regular language

Let $L = \{w \mid w \text{ is of the form } 0^n 1^n, \text{ for all } n \geq 0\}$

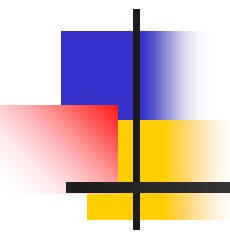
- Hypothesis: L is not regular
- Intuitive rationale: How do you keep track of a running count in an FA?
- A more formal rationale:
 - By contradiction, if L is regular then there should exist a DFA for L .
 - Let k = number of states in that DFA.
 - Consider the special word $w = 0^k 1^k \Rightarrow w \in L$
 - DFA is in some state p_i , after consuming the first i symbols in w



Rationale...

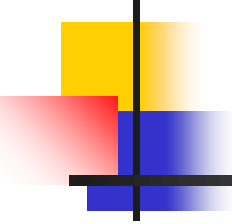
- Let $\{p_0, p_1, \dots, p_k\}$ be the sequence of states that the DFA should have visited after consuming the first k symbols in w which is 0^k
- But there are only k states in the DFA!
- $\implies$ at least one state should repeat somewhere along the path (by  +  Principle)
- $\implies$ Let the repeating state be $p_i = p_j$ for $i < j$
- $\implies$ We can fool the DFA by inputting $0^{(k-(j-i))}1^k$ and still get it to accept (note: $k-(j-i)$ is at most $k-1$).
- $\implies$ DFA accepts strings w/ unequal number of 0s and 1s, implying that the DFA is wrong!





The Pumping Lemma for Regular Languages

A technique that is used to show
that a given language is not
regular



Pumping Lemma for Regular Languages

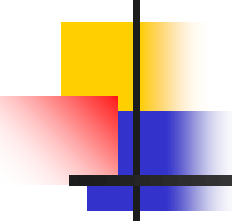
Let L be a regular language.

Then there exists some constant N such that for every string $w \in L$ s.t. $|w| \geq N$, there exists a way to break w into three parts, $w = xyz$, such that:

1. $y \neq \varepsilon$
2. $|xy| \leq N$
3. For all $k \geq 0$, all strings of the form $xy^kz \in L$

This clause should hold for all regular languages.

Definition: N is called the “Pumping Lemma Constant”



Pumping Lemma: Proof

- L is regular $\Rightarrow$ it should have a DFA.
 - Set $N :=$ number of states in the DFA
- Any string $w \in L$, s.t. $|w| \geq N$, should have the form: $w = a_1 a_2 \dots a_m$, where $m \geq N$
- Let the states traversed after reading the first N symbols be: $\{p_0, p_1, \dots, p_N\}$
 - $\Rightarrow$ There are $N+1$ p-states, while there are only N DFA states
 - $\Rightarrow$ at least one state has to repeat
i.e, $p_i = p_j$ where $0 \leq i < j \leq N$ (by PHP)

Pumping Lemma: Proof...

- $\Rightarrow$ We should be able to break $w = \mathbf{x} \mathbf{y} \mathbf{z}$ as follows:

- $\mathbf{x} = a_1 a_2 \dots a_i$; $\mathbf{y} = a_{i+1} a_{i+2} \dots a_j$; $\mathbf{z} = a_{j+1} a_{j+2} \dots a_m$
- $\mathbf{x}$'s path will be $p_0 \dots p_i$
- $\mathbf{y}$'s path will be $p_i p_{i+1} \dots p_j$ (but $p_i = p_j$ implying a loop)
- $\mathbf{z}$'s path will be $p_j p_{j+1} \dots p_m$

- Now consider another string $w_k = \mathbf{x} \mathbf{y}^k \mathbf{z}$, where $k \geq 0$

- Case $k=0$

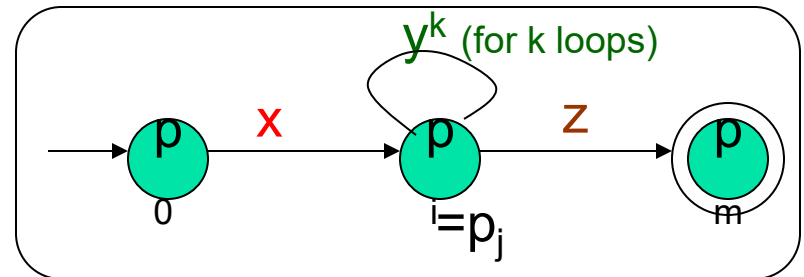
- DFA will reach the accept state p_m

- Case $k > 0$

- DFA will loop for $\mathbf{y}^k$, and finally reach the accept state p_m for $\mathbf{z}$

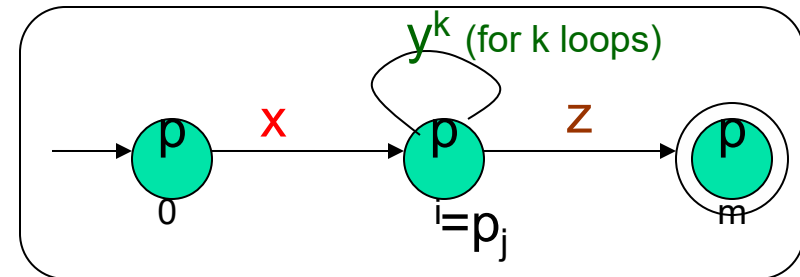
- In either case, $w_k \in L$

This proves part (3) of the lemma



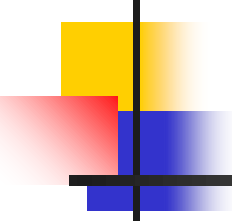
Pumping Lemma: Proof...

- For part (1):
 - Since $i < j$, $y \neq \varepsilon$



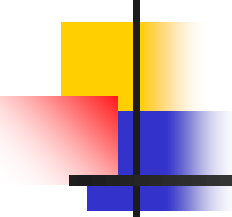
- For part (2):
 - By PHP, the repetition of states has to occur within the first N symbols in w
 - $\implies |xy| \leq N$

□



The Purpose of the Pumping Lemma for RL

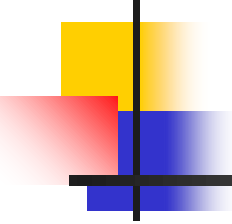
- To prove that some languages *cannot be* regular.



How to use the pumping lemma?

Think of playing a 2 person game

- Role 1: **You** claim that the language cannot be regular
- Role 2: An **adversary** who claims the language is regular
- You show that the adversary's statement will lead to a contradiction that implies pumping lemma *cannot* hold for the language.
- You win!!



How to use the pumping lemma? (The Steps)

1. (you) L is not regular.
 2. (adv.) Claims that L is regular and gives you a value for N as its P/L constant
 3. (you) Using N , choose a string $w \in L$ s.t.,
 1. $|w| \geq N$,
 2. Using w as the template, construct other words w_k of the form xy^kz and show that at least one such $w_k \notin L$

=> this implies you have successfully broken the pumping lemma for the language, and hence that the adversary is wrong.
- (Note: In this process, you may have to try many values of k , starting with $k=0$, and then 2, 3, .. so on, until $w_k \notin L$)

Note: This N can be anything (need not necessarily be the #states in the DFA.
It's the adversary's choice.)

Example of using the Pumping Lemma to prove that a language is not regular

Let $L_{eq} = \{w \mid w \text{ is a binary string with equal number of 1s and 0s}\}$

- Your Claim: L_{eq} is not regular

- Proof:

- By contradiction, let L_{eq} be regular

- P/L constant should exist

- Let N = that P/L constant

- Consider input $w = 0^N 1^N$

(your choice for the template string)

- By pumping lemma, we should be able to break $w = xyz$, such that:

- 1) $y \neq \epsilon$

- 2) $|xy| \leq N$

- 3) For all $k \geq 0$, the string xy^kz is also in L

adv.

adv.

you

yo
u

Template string $w = 0^N 1^N = \underbrace{00 \dots 0}_N \cdot \underbrace{011 \dots 1}_N$

Proof...

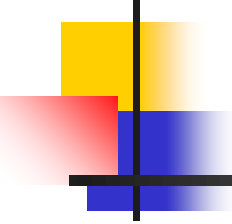
- Because $|xy| \leq N$, xy should contain only 0s
 - (This and because $y \neq \varepsilon$, implies $y = 0^+$)
- Therefore x can contain *at most* $N-1$ 0s
- Also, all the N 1s must be inside z
- By (3), any string of the form $xy^kz \in L_{eq}$ for all $k \geq 0$
- Case $k=0$: xz has at most $N-1$ 0s but has N 1s
- Therefore, $xy^0z \notin L_{eq}$
- This violates the P/L (a contradiction) ↯

Setting $k=0$ is referred to as **"pumping down"**

Setting $k>1$ is referred to as **"pumping up"**

Another way of proving this will be to show that if the #0s is arbitrarily pumped up (e.g., $k=2$), then the #0s will become exceed the #1s

□
you



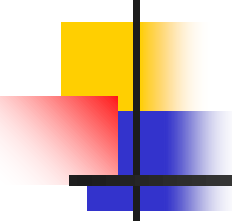
Exercise 2

Prove $L = \{0^n 1 0^n \mid n \geq 1\}$ is not regular

Note: This n is not to be confused with the pumping lemma constant N . That *can* be different.

In other words, the above question is same as proving:

- $L = \{0^m 1 0^m \mid m \geq 1\}$ is not regular

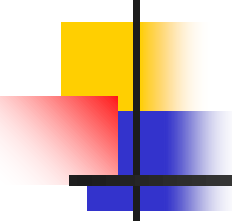


Example 3: Pumping Lemma

Claim: $L = \{ 0^i \mid i \text{ is a perfect square} \}$ is not regular

■ Proof:

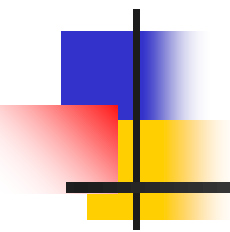
- By contradiction, let L be regular.
- P/L should apply
- Let $N = P/L$ constant
- Choose $w = 0^{N^2}$
- By pumping lemma, $w = xyz$ satisfying all three rules
- By rules (1) & (2), y has between 1 and N 0s
- By rule (3), any string of the form xy^kz is also in L for all $k \geq 0$
- Case $k=0$:
 - $\#zeros(xy^0z) = \#zeros(xyz) - \#zeros(y)$
 - $N^2 - N \leq \#zeros(xy^0z) \leq N^2 - 1$
 - $(N-1)^2 < N^2 - N \leq \#zeros(xy^0z) \leq N^2 - 1 < N^2$
 - $xy^0z \notin L$
 - But the above will complete the proof ONLY IF $N > 1$.
 - ... (proof contd.. Next slide)



Example 3: Pumping Lemma

- (proof contd...)
 - If the adversary pick $N=1$, then $(N-1)^2 \leq N^2 - N$, and therefore the #zeros(xy^0z) could end up being a perfect square!
 - This means that pumping down (i.e., setting $k=0$) is not giving us the proof!
 - So lets try pumping up next...
- Case $k=2$:
 - #zeros (xy^2z) = #zeros (xyz) + #zeros (y)
 - $N^2 + 1 \leq \text{\#zeros}(xy^2z) \leq N^2 + N$
 - $N^2 < N^2 + 1 \leq \text{\#zeros}(xy^2z) \leq N^2 + N < (N+1)^2$
 - $xy^2z \notin L$ 
- (Notice that the above should hold for all possible N values of $N>0$. Therefore, this completes the proof.)

Closure properties of Regular Languages

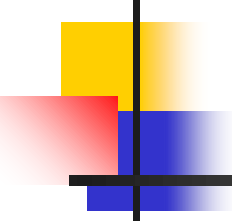


Closure properties for Regular Languages (RL)

This is different from Kleene closure

- Closure property:
 - If a set of regular languages are combined using an operator, then the resulting language is also regular.
- Regular languages are closed under:
 - Union, intersection, complement, difference
 - Reversal
 - Kleene closure
 - Concatenation
 - Homomorphism
 - Inverse homomorphism

Now, let's prove all of this!

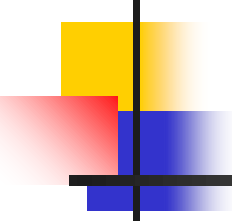


RLs are closed under union

- IF L and M are two RLs THEN:
 - they both have two corresponding regular expressions, R and S respectively
 - $(L \cup M)$ can be represented using the regular expression $R+S$
 - Therefore, $(L \cup M)$ is also regular



How can this be proved using FAs?



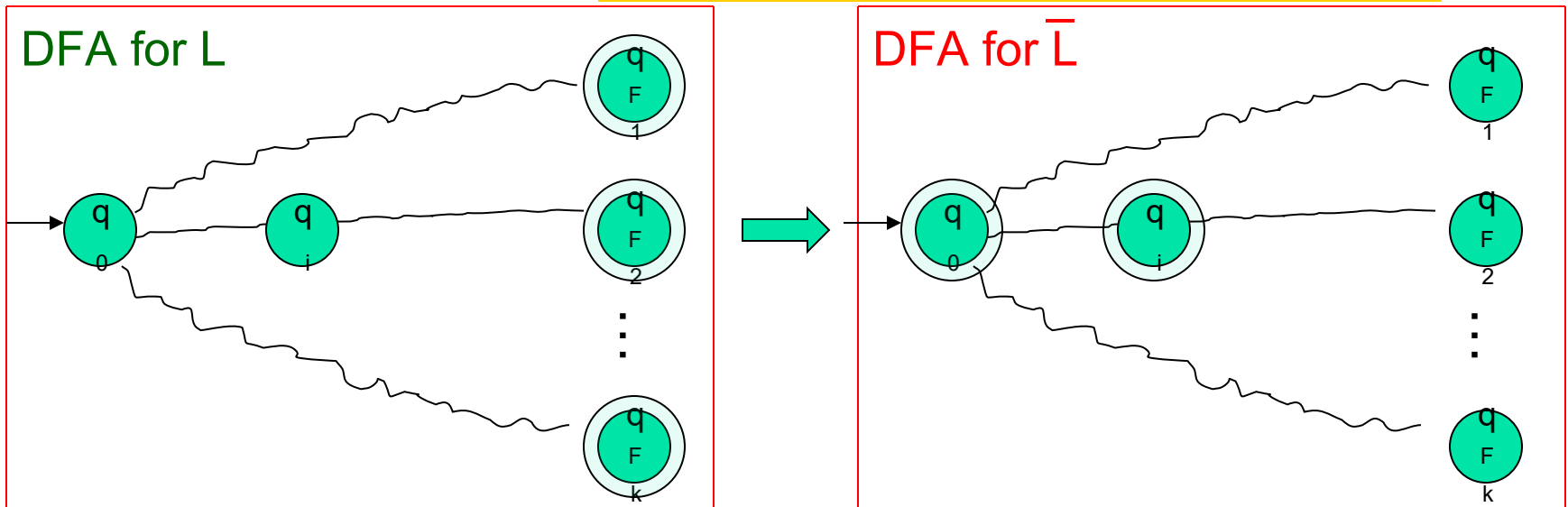
Closure Under Concatenation and Kleene Closure

- Same idea:
 - RS is a regular expression whose language is LM .
 - R^* is a regular expression whose language is L^* .

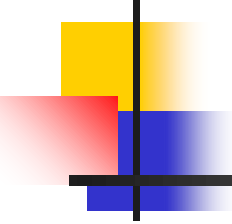
RLs are closed under complementation

- If L is an RL over Σ , then $\bar{L} = \Sigma^* - L$
 - To show $\bar{L}$ is also regular, make the following construction

Convert every final state into non-final, and every non-final state into a final state

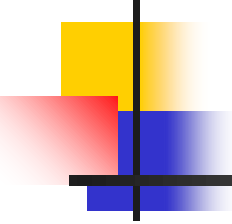


Assumes q_0 is a non-final state. If not, do the opposite.



RLs are closed under intersection

- A quick, indirect way to prove:
 - By DeMorgan's law:
 - $L \cap M = \overline{(\bar{L} \cup \bar{M})}$
 - Since we know RLs are closed under union and complementation, they are also closed under intersection
- A more direct way would be construct a finite automaton for $L \cap M$

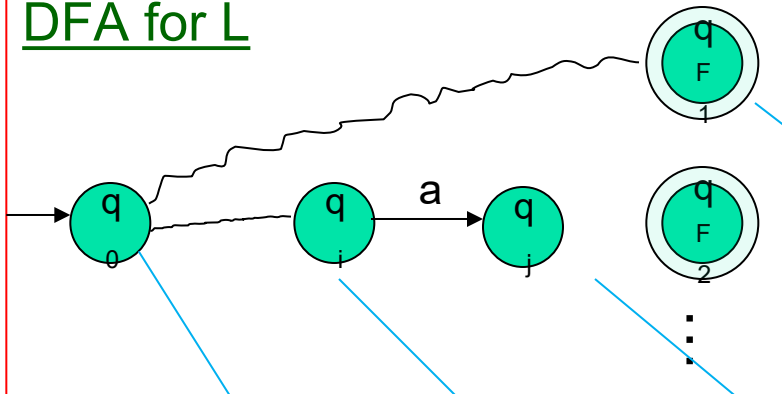


DFA construction for $L \cap M$

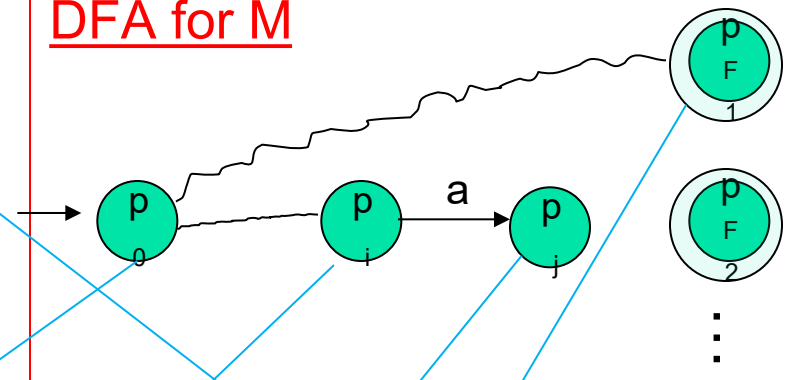
- $A_L = \text{DFA for } L = \{Q_L, \Sigma, q_L, F_L, \Delta_L\}$
- $A_M = \text{DFA for } M = \{Q_M, \Sigma, q_M, F_M, \Delta_M\}$
- Build $A_{L \cap M} = \{Q_L \times Q_M, \Sigma, (q_L, q_M), F_L \times F_M, \Delta\}$ such that:
 - $\Delta((p, q), a) = (\Delta_L(p, a), \Delta_M(q, a))$, where p in Q_L , and q in Q_M
- This construction ensures that a string w will be accepted if and only if w reaches an accepting state in both input DFAs.

DFA construction for $L \cap M$

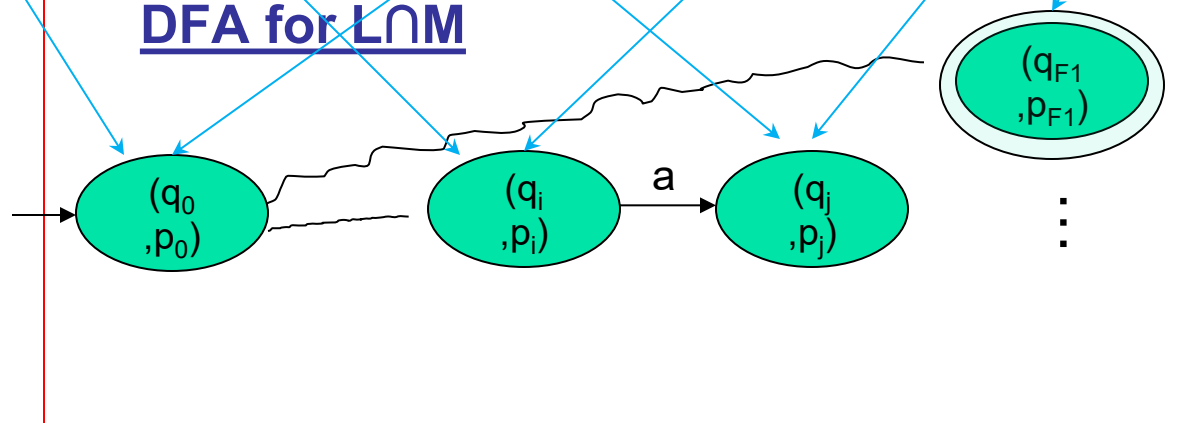
DFA for L



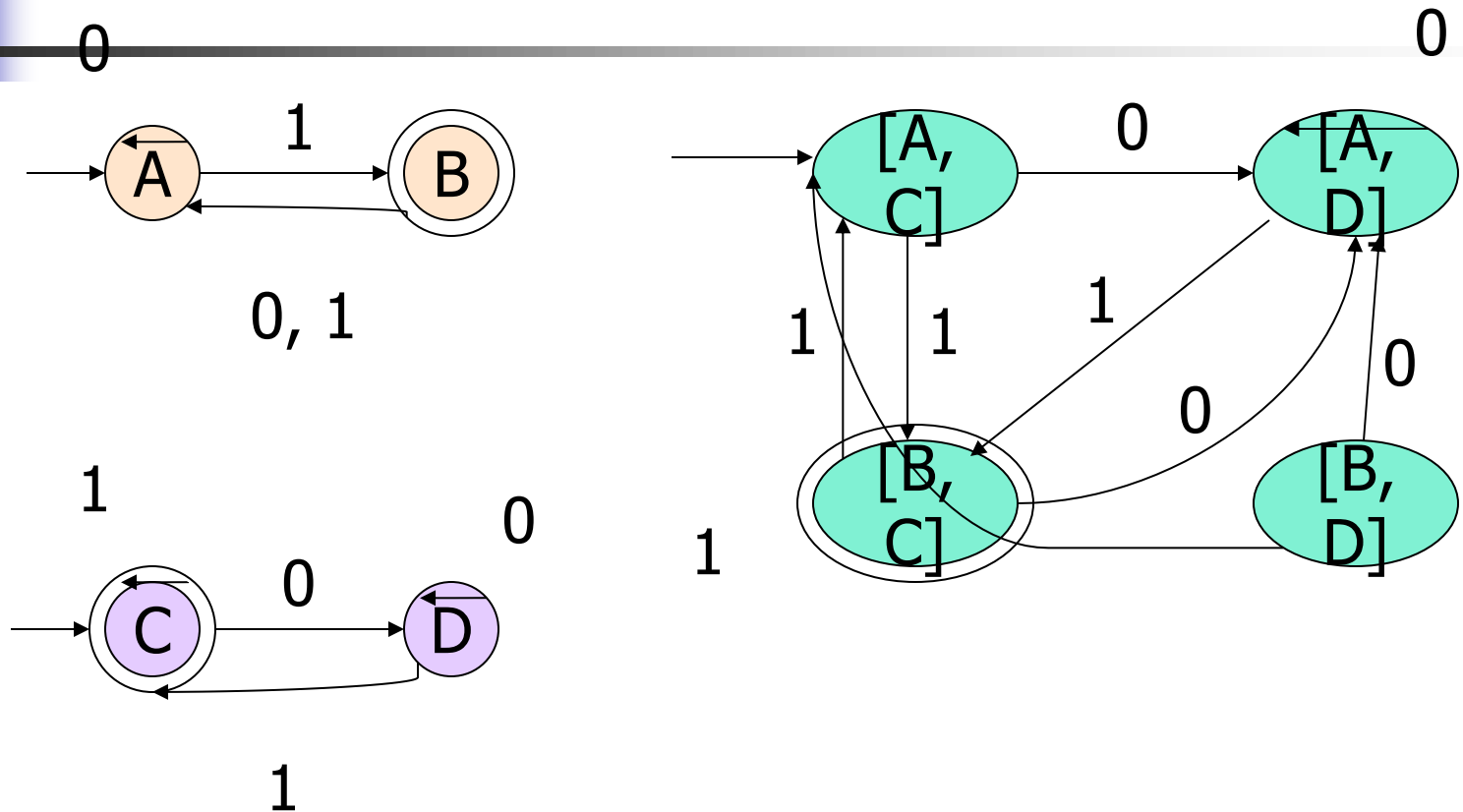
DFA for M



DFA for $L \cap M$



Example: Product DFA for Intersection



RLs are closed under set difference

- We observe:

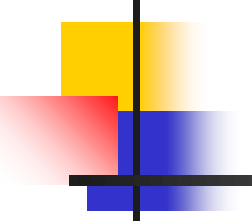
- $L - M = L \cap \overline{M}$

Closed under intersection

Closed under
complementation

- Therefore, $L - M$ is also regular

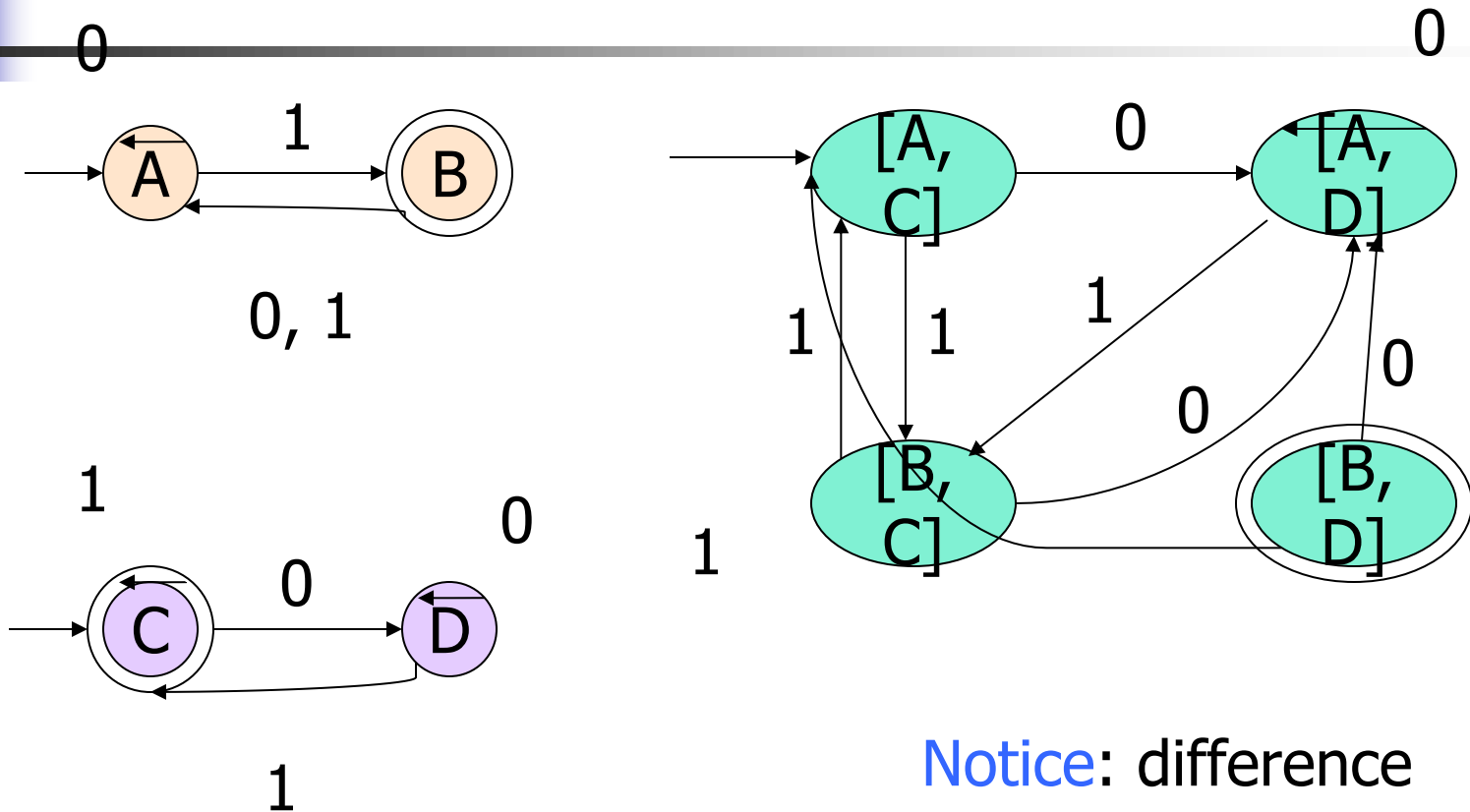
How can this be proved using FAs?



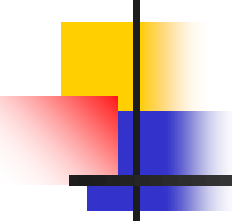
DFA construction for $L \cap M$

- Let A and B be DFA's whose languages are L and M , respectively.
- Construct another DFA C , the product automaton of A and B .
- Make the final states of C be the pairs where A -state is final but B -state is not.

Example: Product DFA for Difference



Notice: difference is the empty language



RLs are closed under reversal

Reversal of a string w is denoted by w^R

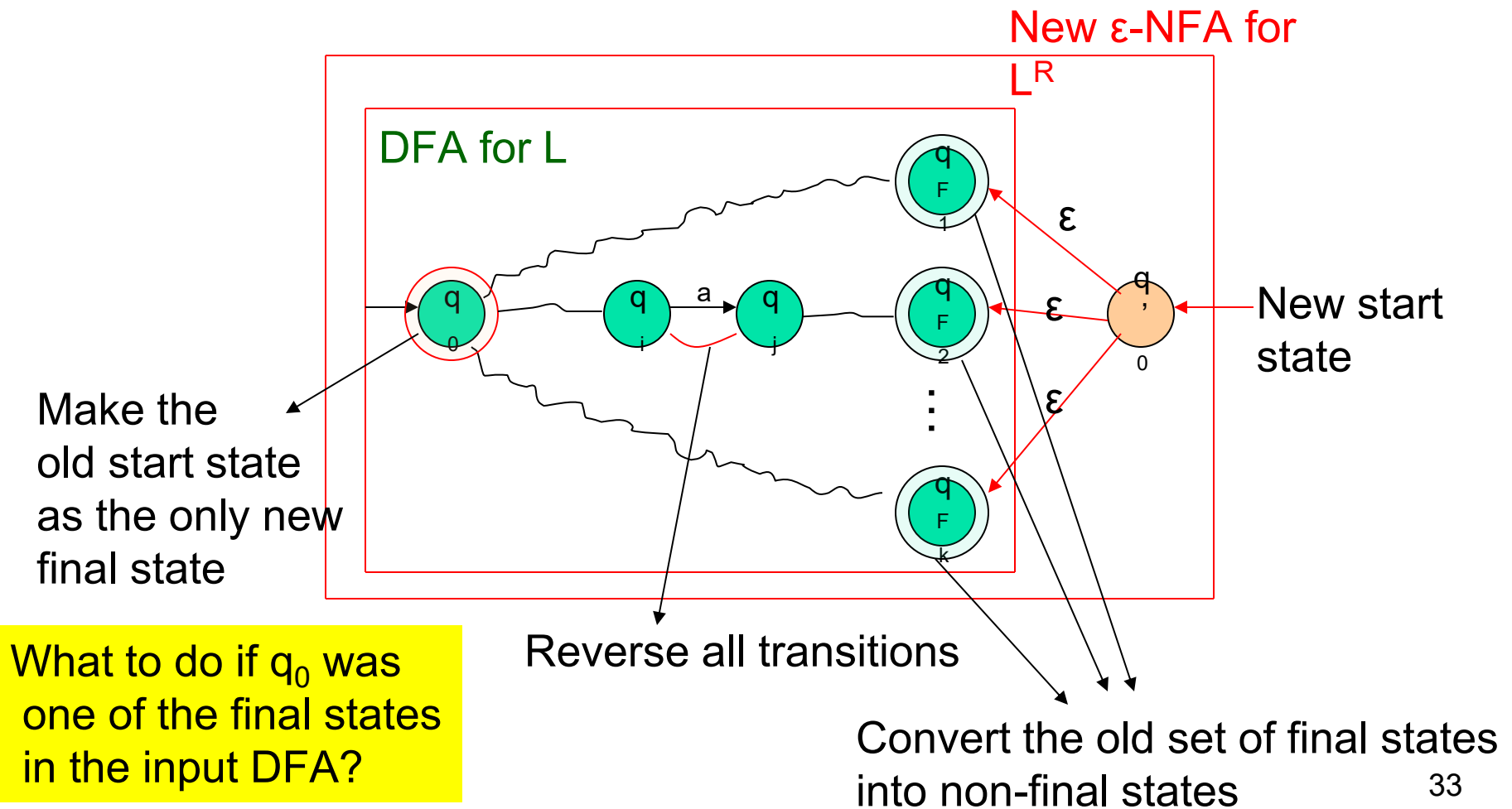
- E.g., $w=00111$, $w^R=11100$

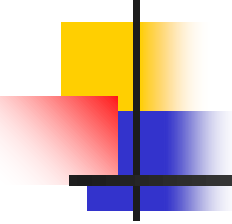
Reversal of a language:

- L^R = The language generated by reversing all strings in L

Theorem: If L is regular, then L^R is also regular

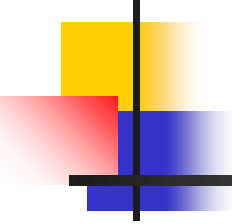
ϵ -NFA Construction for L^R





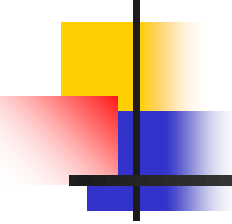
If L is regular, L^R is regular (proof using regular expressions)

- Let E be a regular expression for L
- Given E , how to build E^R ?
- Basis: If $E = \varepsilon$, $\emptyset$, or a , then $E^R = E$
- Induction: Every part of E (refer to the part as “ F ”) can be in only *one* of the three following forms:
 1. $F = F_1 + F_2$
 - $F^R = F_1^R + F_2^R$
 2. $F = F_1 F_2$
 - $F^R = F_2^R F_1^R$
 3. $F = (F_1)^*$
 - $(F^R)^* = (F_1^R)^*$



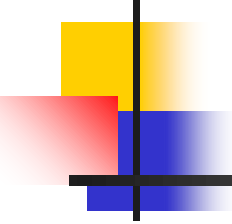
Example: Reversal of a RE

- Let $E = 01^* + 10^*$.
- $E^R = (01^* + 10^*)^R = (01^*)^R + (10^*)^R$
- $= (1^*)^R 0^R + (0^*)^R 1^R$
- $= (1^R)^* 0 + (0^R)^* 1$
- $= 1^* 0 + 0^* 1.$



Homomorphisms

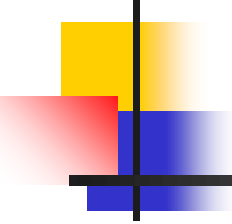
- Substitute each symbol in Σ (main alphabet) by a corresponding string in T (another alphabet)
 - $h: \Sigma \rightarrow T^*$
- Example:
 - Let $\Sigma = \{0, 1\}$ and $T = \{a, b\}$
 - Let a homomorphic function h on Σ be:
 - $h(0) = ab, h(1) = \varepsilon$
 - If $w = 10110$, then $h(w) = \varepsilon ab \varepsilon \varepsilon ab = abab$
- In general,
 - $h(w) = h(a_1) h(a_2) \dots h(a_n)$



Example: Closure under Homomorphism

- Let $h(0) = ab$; $h(1) = \varepsilon$.
- Let L be the language of regular expression $01^* + 10^*$.
- Then $h(L)$ is the language of regular expression $ab\varepsilon^* + \varepsilon(ab)^*$.

Note: use parentheses to enforce the proper grouping.



Example – Continued

- $\mathbf{ab}\varepsilon^* + \varepsilon(\mathbf{ab})^*$ can be simplified.
- $\varepsilon^* = \varepsilon$, so $\mathbf{ab}\varepsilon^* = \mathbf{ab}\varepsilon$.
- ε is the identity under concatenation.
 - That is, $\varepsilon E = E\varepsilon = E$ for any RE E .
- Thus, $\mathbf{ab}\varepsilon^* + \varepsilon(\mathbf{ab})^* = \mathbf{ab}\varepsilon + \varepsilon(\mathbf{ab})^* = \mathbf{ab} + (\mathbf{ab})^*$.
- Finally, $L(\mathbf{ab})$ is contained in $L((\mathbf{ab})^*)$, so a RE for $h(L)$ is $(\mathbf{ab})^*$.

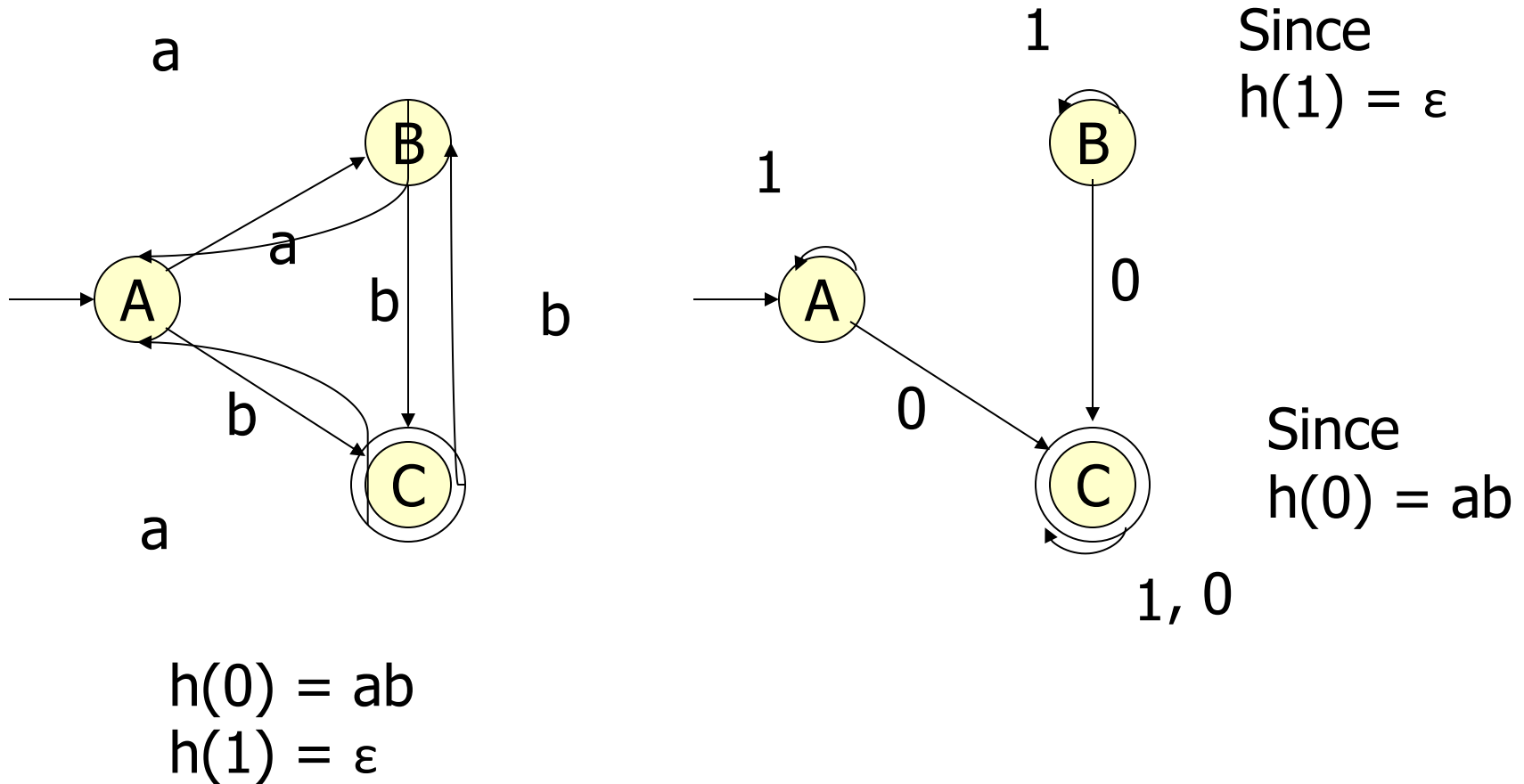
Inverse homomorphism

- Let $h: \Sigma \rightarrow T^*$
- Let M be a language over alphabet T
- $h^{-1}(M) = \{w \mid w \in \Sigma^* \text{ s.t.}, h(w) \in M\}$

Claim: *If M is regular, then so is $h^{-1}(M)$*

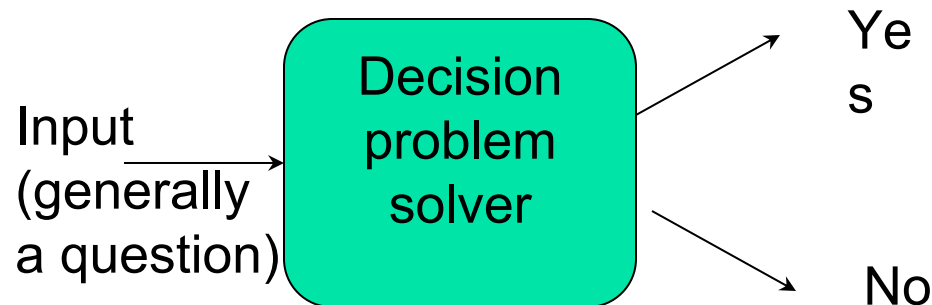
- Proof:
 - Let A be a DFA for M
 - Construct another DFA A' which encodes $h^{-1}(M)$
 - A' is an exact replica of A , except that its transition functions are s.t. for any input symbol a in Σ , A' will simulate $h(a)$ in A .
 - $\Delta(p, a) = \Delta(p, h(a))$

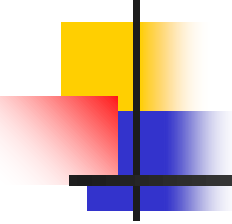
Example: Inverse Homomorphism Construction



Decision properties of regular languages

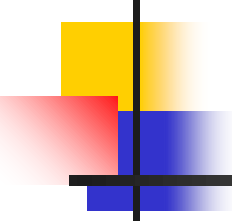
Any “decision problem” looks like this:





Membership question

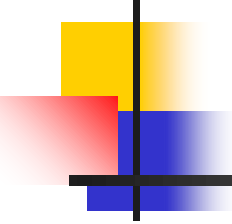
- Decision Problem: Given L , is w in L ?
- Possible answers: Yes or No
- Approach:
 1. Build a DFA for L
 2. Input w to the DFA
 3. If the DFA ends in an accepting state, then yes; otherwise no.



Emptiness test

- Decision Problem: Is $L = \emptyset$?
- Approach:
On a DFA for L:
 1. From the start state, run a *reachability* test, which returns:
 1. success: if there is at least one final state that is reachable from the start state
 2. failure: otherwise
 2. $L = \emptyset$ if and only if the reachability test fails

How to implement the reachability test?



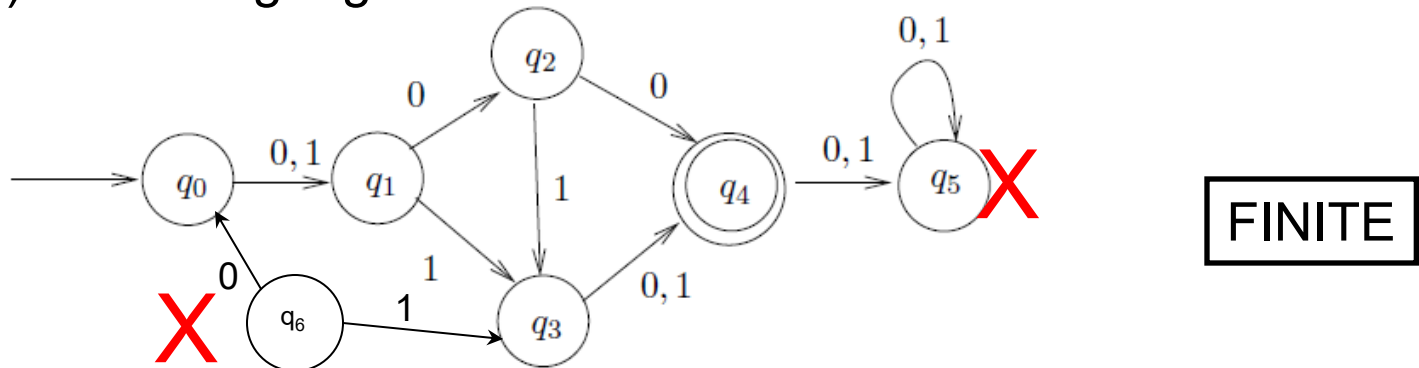
Finiteness

- Decision Problem: Is L finite or infinite?
- Approach:
 - On a DFA for L:
 1. Remove all states unreachable from the start state
 2. Remove all states that cannot lead to any accepting state.
 3. After removal, check for cycles in the resulting FA
 4. L is finite if there are no cycles; otherwise it is infinite
- Another approach
 - Build a regular expression and look for Kleene closure

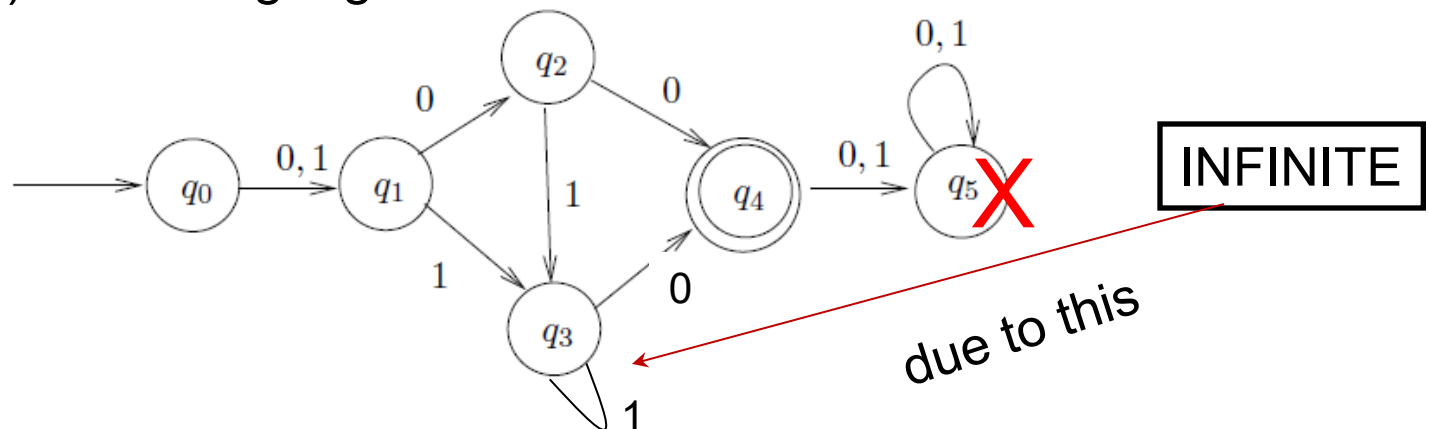
How to implement steps 2 and 3?

Finiteness test - examples

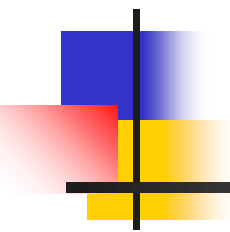
Ex 1) Is the language of this DFA finite or infinite?

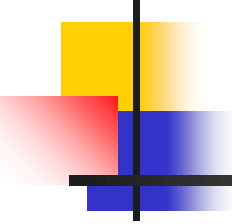


Ex 2) Is the language of this DFA finite or infinite?



Equivalence & Minimization of DFAs





Applications of interest

- Comparing two DFAs:
 - $L(\text{DFA}_1) == L(\text{DFA}_2)$?

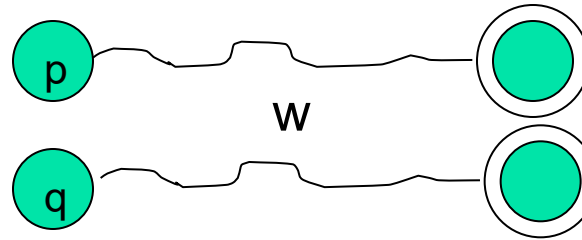
- How to minimize a DFA?
 1. Remove unreachable states
 2. Identify & condense equivalent states into one

When to call two states in a DFA “equivalent”?

Past doesn't matter - only future does!

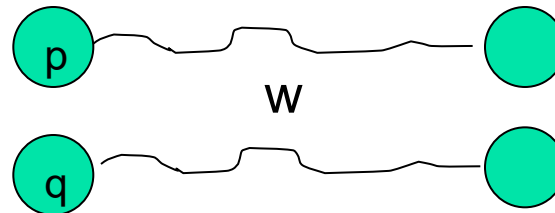
Two states p and q are said to be *equivalent* iff:

- i) Any string w accepted by starting at p is also accepted by starting at q ;



AND

- i) Any string w rejected by starting at p is also rejected by starting at q .

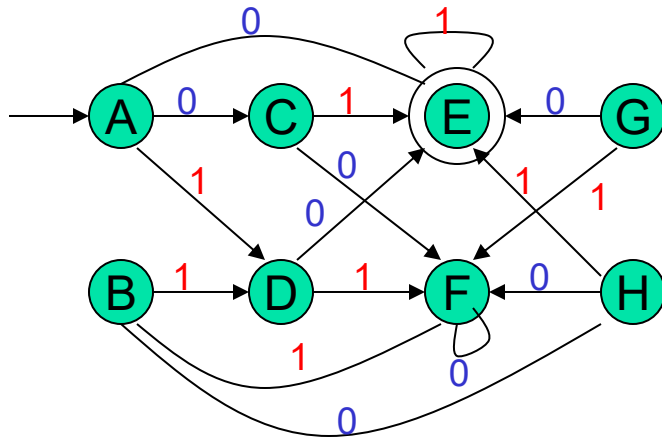


□

$p \equiv q$

Computing equivalent states in a DFA

Table Filling Algorithm



Pass #0

1. Mark accepting states $\neq$ non-accepting states

Pass #1

1. Compare every pair of states
2. Distinguish by one symbol transition
3. Mark = or $\neq$ or blank(tbd)

Pass #2

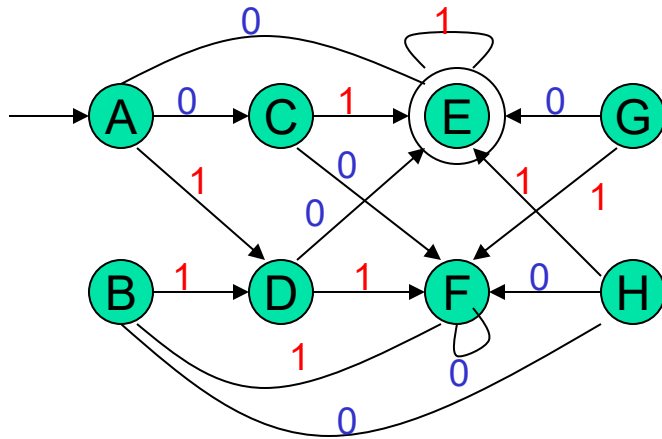
1. Compare every pair of states
2. Distinguish by up to two symbol transitions (until different or same or tbd)

....

(keep repeating until table complete)

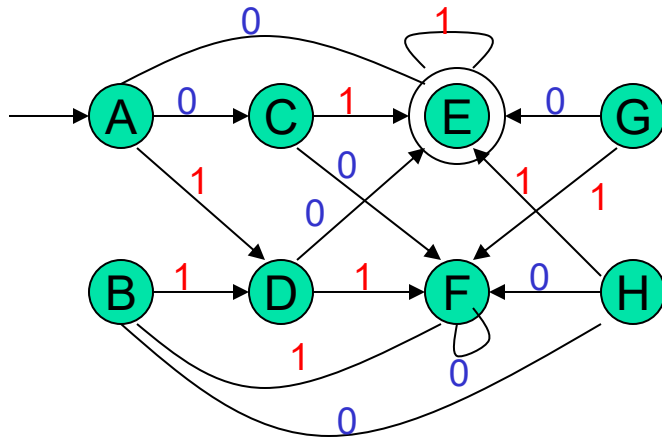
A	=							
B	=	=						
C	x	x	=					
D	x	x	x	=				
E	x	x	x	x	=			
F	x	x	x	x	x	=		
G	x	x	x	=	x	x	=	
H	x	x	=	x	x	x	x	=
	A	B	C	D	E	F	G	H

Table Filling Algorithm - step by step



A	=							
B		=						
C			=					
D				=				
E					=			
F						=		
G							=	
H								=
	A	B	C	D	E	F	G	H

Table Filling Algorithm - step by step



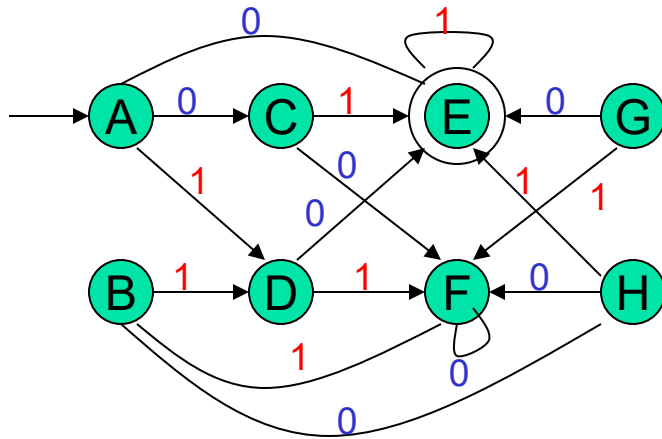
1. Mark **X** between accepting vs. non-accepting state



A	=							
B		=						
C			=					
D				=				
E	X	X	X	X	=			
F					X	=		
G					X		=	
H					X			=
	A	B	C	D	E	F	G	H



Table Filling Algorithm - step by step

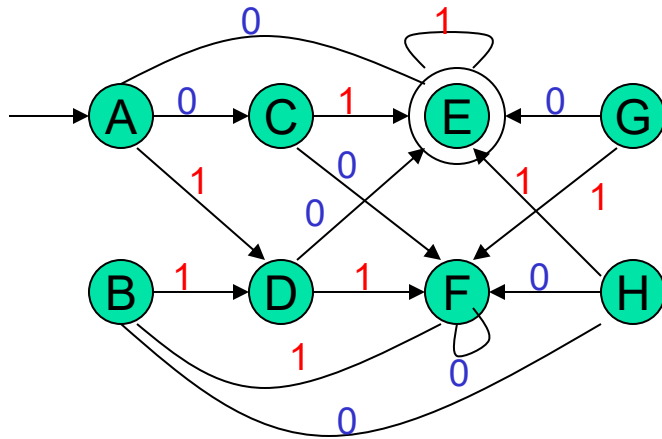


1. Mark X between accepting vs. non-accepting state
2. Look 1- hop away for distinguishing states or strings

A	=							
B		=						
C	X		=					
D	X			=				
E	X	X	X	X	=			
F					X	=		
G	X				X		=	
H	X				X			=
	A	B	C	D	E	F	G	H



Table Filling Algorithm - step by step

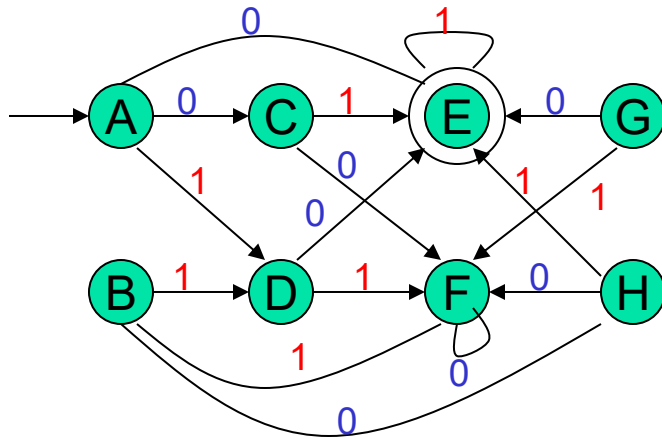


1. Mark X between accepting vs. non-accepting state
2. Look 1- hop away for distinguishing states or strings

A	=							
B		=						
C	X	X	=					
D	X	X		=				
E	X	X	X	X	=			
F					X	=		
G	X	X			X		=	
H	X	X			X			=
	A	B	C	D	E	F	G	H



Table Filling Algorithm - step by step

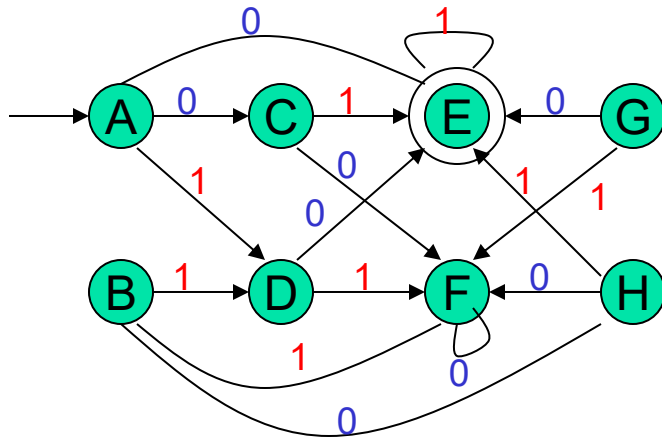


1. Mark X between accepting vs. non-accepting state
2. Look 1- hop away for distinguishing states or strings

A	=							
B		=						
C	X	X	=					
D	X	X	X	=				
E	X	X	X	X	=			
F			X		X	=		
G	X	X	X		X		=	
H	X	X	=		X			=
	A	B	C	D	E	F	G	H



Table Filling Algorithm - step by step

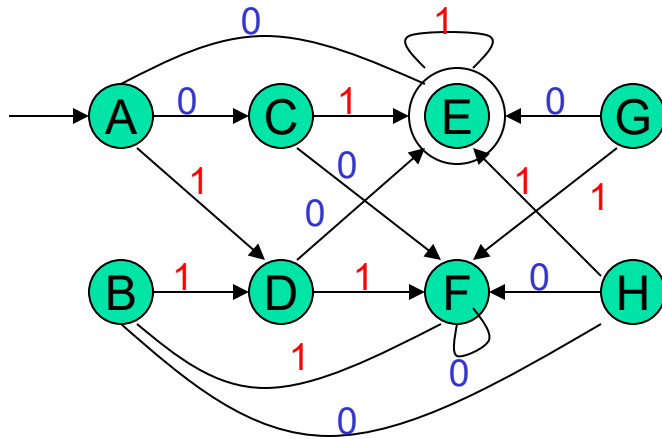


1. Mark **X** between accepting vs. non-accepting state
2. Look 1- hop away for distinguishing states or strings

A	=							
B		=						
C	X	X	=					
D	X	X	X	=				
E	X	X	X	X	=			
F			X	X	X	=		
G	X	X	X	=	X		=	
H	X	X	=	X	X			=
	A	B	C	D	E	F	G	H



Table Filling Algorithm - step by step

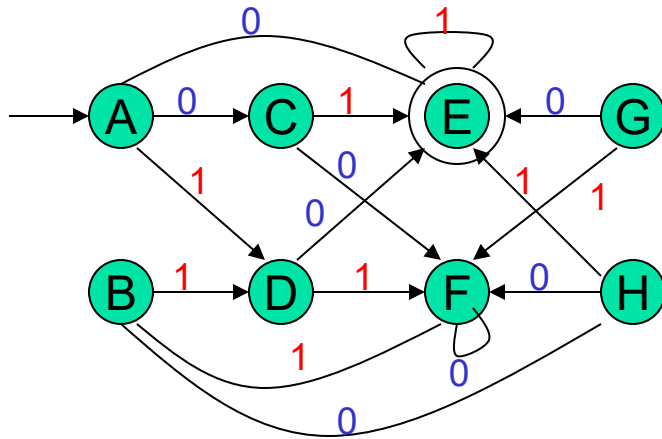


1. Mark X between accepting vs. non-accepting state
2. Look 1- hop away for distinguishing states or strings

A	=							
B		=						
C	X	X	=					
D	X	X	X	=				
E	X	X	X	X	=			
F			X	X	X	=		
G	X	X	X	=	X	X	=	
H	X	X	=	X	X	X		=
	A	B	C	D	E	F	G	H



Table Filling Algorithm - step by step

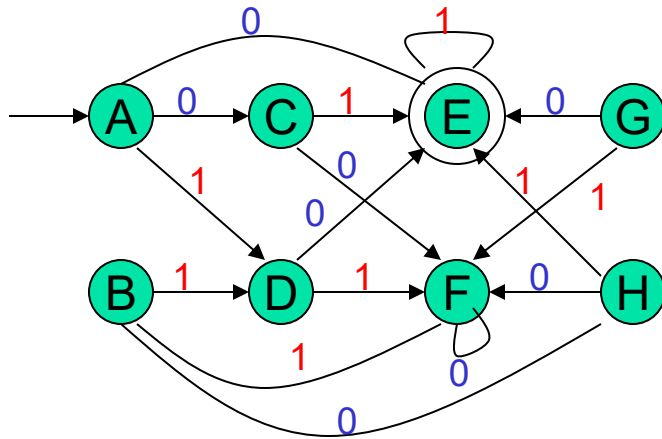


1. Mark **X** between accepting vs. non-accepting state
2. Look 1- hop away for distinguishing states or strings

A	=							
B		=						
C	X	X	=					
D	X	X	X	=				
E	X	X	X	X	=			
F			X	X	X	=		
G	X	X	X	=	X	X	=	
H	X	X	=	X	X	X	X	=
	A	B	C	D	E	F	G	H



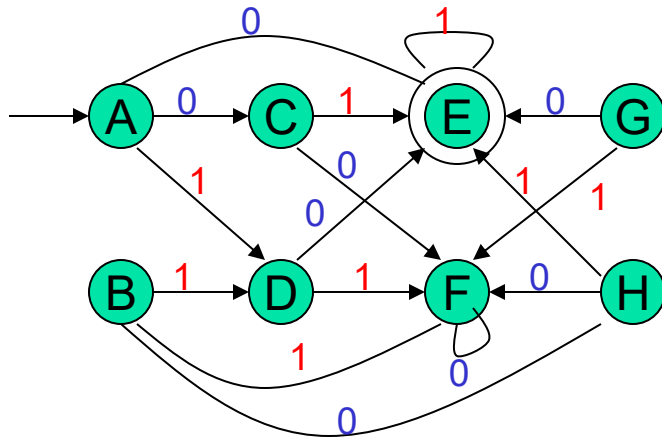
Table Filling Algorithm - step by step



A	=							
B	=	=						
C	X	X	=					
D	X	X	X	=				
E	X	X	X	X	=			
F	X	X	X	X	X	=		
G	X	X	X	=	X	X	=	
H	X	X	=	X	X	X	X	=
	A	B	C	D	E	F	G	H

1. Mark **X** between accepting vs. non-accepting state
2. Pass 1:
Look 1- hop away for distinguishing states or strings
3. Pass 2:
Look 1-hop away again for distinguishing states or strings
continue....

Table Filling Algorithm - step by step



A	=							
B	=	=						
C	X	X	=					
D	X	X	X	=				
E	X	X	X	X	=			
F	X	X	X	X	X	=		
G	X	X	X	=	X	X	=	
H	X	X	=	X	X	X	X	=
	A	B	C	D	E	F	G	H

1. Mark **X** between accepting vs. non-accepting state

2. Pass 1:

Look 1- hop away for distinguishing states or strings

3. Pass 2:

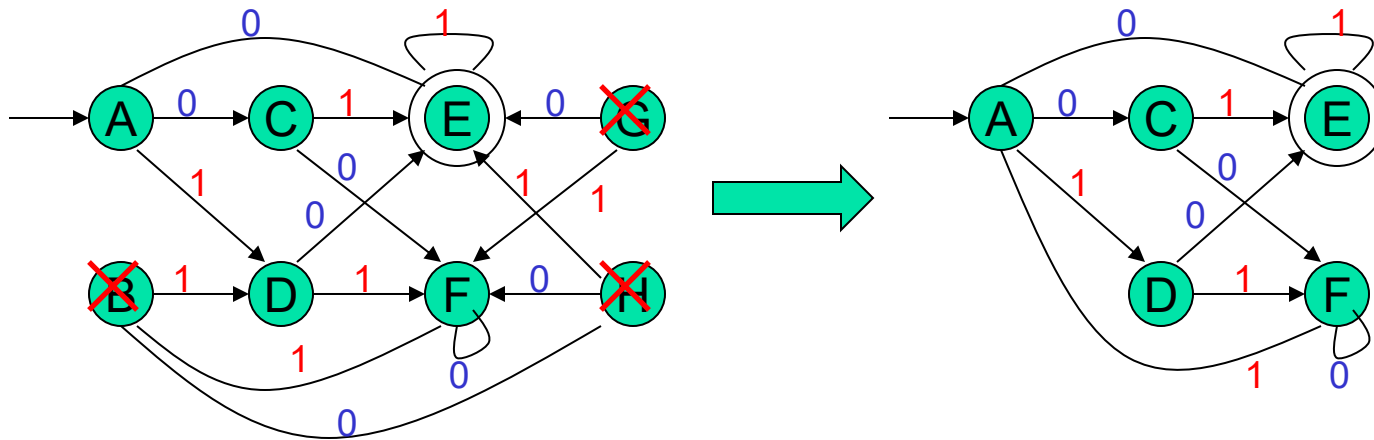
Look 1-hop away again for distinguishing states or strings

continue....

Equivalences:

- A=B
- C=H
- D=G

Table Filling Algorithm - step by step

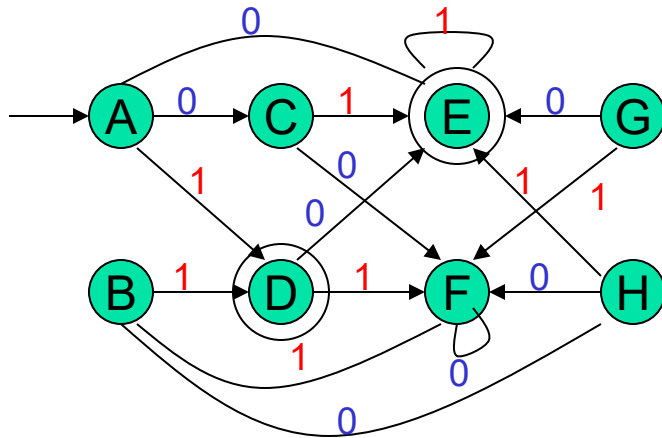


Retrain only one copy for
each equivalence set of states

Equivalences:

- A=B
- C=H
- D=G

Table Filling Algorithm – special case



A	=							
B		=						
C			=					
D				=				
E				?	=			
F						=		
G							=	
H								=
	A	B	C	D	E	F	G	H

Q) What happens if the input DFA has more than one final state?
 Can all final states initially be treated as equivalent to one another?

Putting it all together ...

How to minimize a DFA?

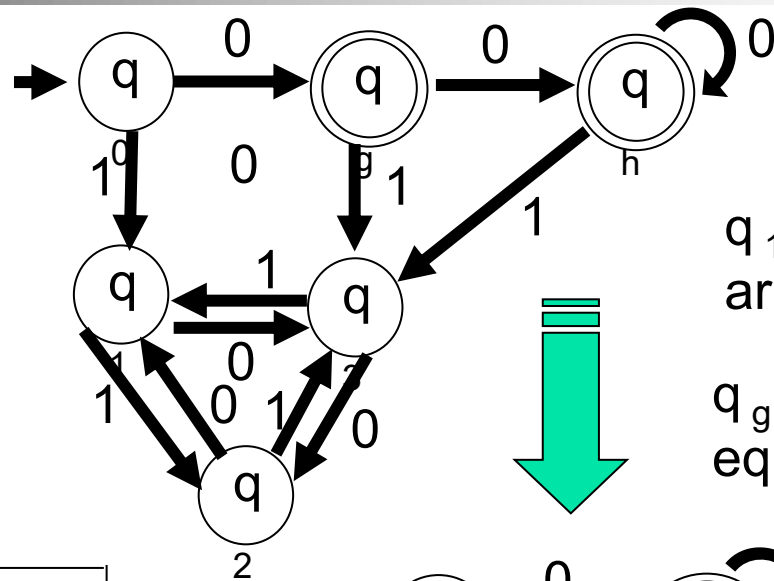
- Goal: Minimize the number of states in a DFA
- Algorithm:
 - 1. Eliminate states unreachable from the start state
 - 2. Identify and remove equivalent states
 - 3. Output the resultant DFA

Depth-first traversal from the start state

Table filling
algorithm

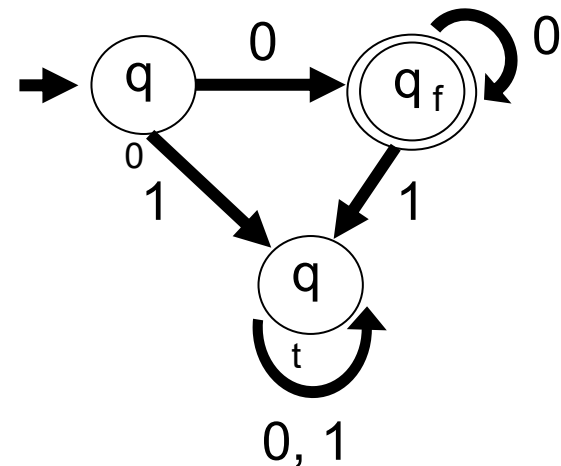
Example: Minimization of DFA M

q	D[i, j]				
0					
q	X				
1					
q	X	=			
2					
q	X	=	=		
3					
q	X	X	X	X	
g					
q	X	X	X	X	=
h					
	q	q	q	q	q
	0	1	2	3	g

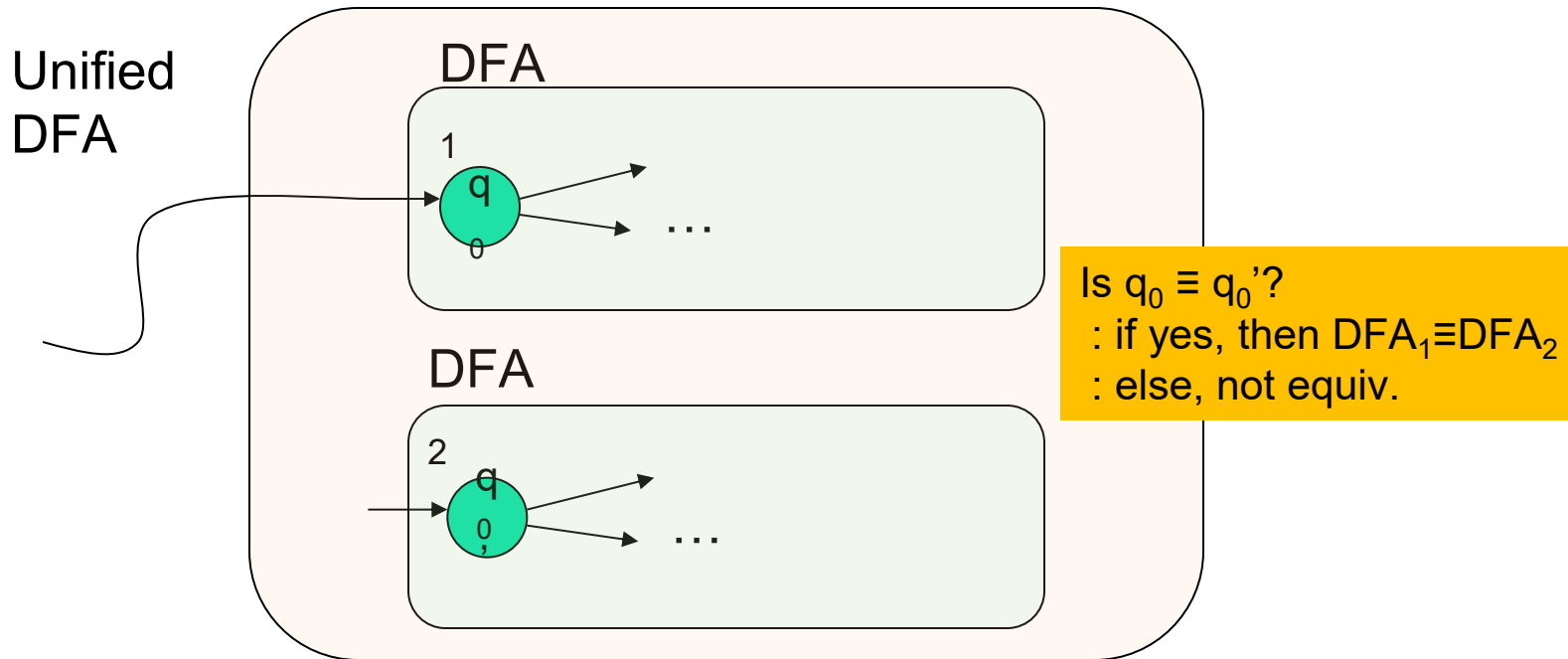


q₁, q₂, and q₃ are equivalent

q_g and q_h are equivalent



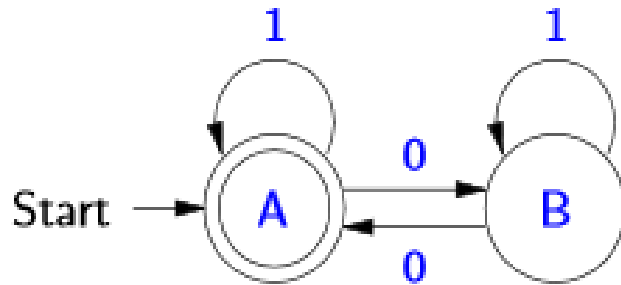
Are Two DFAs Equivalent?



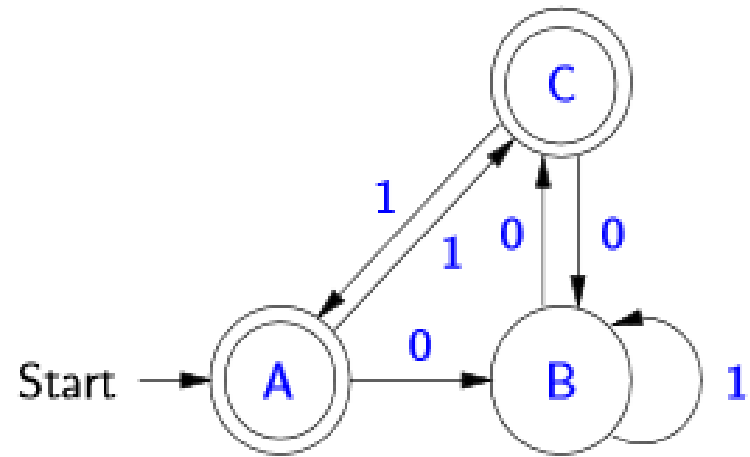
1. Make a new dummy DFA by just putting together both DFAs
2. Run table-filling algorithm on the unified DFA
3. IF the start states of both DFAs are found to be equivalent,
 THEN: $DFA_1 \equiv DFA_2$
 ELSE: different

Example: Equivalency Check

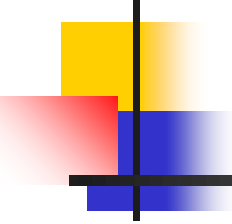
Problem: Are they equivalent?



DFA - A



DFA - B



Summary

- How to prove languages are not regular?
 - Pumping lemma & its applications
- Closure properties of regular languages
- Simplification of DFAs
 - How to remove unreachable states?
 - How to identify and collapse equivalent states?
 - How to minimize a DFA?
 - How to tell whether two DFAs are equivalent?